

Федеральное государственное автономное образовательное учреждение
высшего образования
Московский физико-технический институт
(национальный исследовательский университет)
Физтех-школа радиотехники и компьютерных технологий
Кафедра технологий разработки и программирования микропроцессорных систем

Методы верификации кэша второго уровня графического процессора общего назначения

Обучающийся
Кулевич Анастасия Юрьевна

Научный консультант
Маслов Максим Владимирович

Научный руководитель
Козлов Вячеслав Константинович

Москва 2026

Аннотация

Разработка Систем-на-Кристалле (СнК) является крайне дорогостоящим процессом. С каждым последующим этапом разработки стоимость исправления ошибки только увеличивается, поэтому разработчики интегральных схем стремятся обнаружить их как можно раньше. Поэтому задача верификации состоит не только в подтверждении соответствия итогового продукта заданным требованиям с высокой степенью уверенности, но и разумные сроки процесса подтверждения. Возникает вопрос о выборе метода верификации, который наилучшим образом соответствует заданным требованиям.

Целью данной работы является исследование современных методов функциональной верификации и их сравнительный анализ в контексте задачи верификации кэша второго уровня графического процессора общего назначения (GPGPU).

В работе приводится описание формальных методов и методов, основанных на симуляции. Излагаются основные концепции, на которых базируются вышеупомянутые методы; на основе этого выделяются соответствующие преимущества и недостатки. Осуществляется реализация этих методов по отношению к кэшу второго уровня GPGPU, а также сравнение методов по критериям применимости, эффективности и качества.

Содержание

1	Введение	4
1.1	Цель работы	5
2	Методы, основанные на симуляции	6
2.1	Направленное тестирование	6
2.2	Ограниченное случайное тестирование	7
2.2.1	UVM	7
2.2.2	Функциональное и кодовое покрытие	8
2.3	Трассы	8
2.4	Ключевые особенности	9
3	Формальная верификация	10
3.1	Базовые принципы работы	10
3.2	Проблема комбинаторного взрыва	11
3.2.1	Бинарная диаграмма решений	11
3.2.2	Задача выполнимости булевых формул	12
3.2.3	Редукция конуса влияния	13
3.3	Формальное покрытие	14
4	Используемые инструменты	15
4.1	JasperGold	15
4.2	SystemVerilog Assertions (SVA)	15
4.3	Характеристики машины	16
5	Практическое применение методов верификации к L2 кэшу GPGPU	17
5.1	D-stage	18
5.2	C-stage	23
5.3	DD-stage	25
5.4	T-stage	27
5.5	Q-stage	28
6	Сравнение примененных методов верификации	31
6.1	Прогресс верификации	31
6.2	Качество верификации	32
6.3	Исправление ошибок	32
6.4	Порог входа	33
6.5	Применение	33

1. Введение

В контексте данной работы под верификацией будем иметь ввиду функциональную верификацию, то есть проверку соответствия имплементации логической схемы её спецификации. Имплементация, как правило, представляет RTL-код (Register Transfer Level). Спецификация написана на естественном (используемом людьми) языке, поэтому для верификации устройства её необходимо интерпретировать, то есть переписать в компьютерный код так, чтобы тем или иным способом выразить архитектурное намерение. Обратим внимание, что под функциональной верификацией мы понимаем не только динамические методы, но и статические (такие как формальная верификация).

Выделим некоторые вызовы функциональной верификации:

1. Количество состояний системы.

Как известно, сложность интегральных схем стремительно растёт[moore], но вместе с этим растёт и сложность их верификации. Более того, предполагается, что сложность верификации может расти экспоненциально по отношению к сложности устройства[appr_and_chall]. Эта проблема становится еще более явной, если учесть, что различные верификационные активности занимают в среднем от 50% до 70% всего времени разработки проекта[trends].

2. С помощью функциональной верификации мы можем установить только существование ошибки, но не ее отсутствие.

Иначе говоря, отсутствует способ доказать полную корректность работы устройства. Частично это следствие из предыдущей проблемы: с ростом числа логических элементов становится невозможным проверить все возможные состояния системы за ограниченное время. Важно и то, что сама спецификация и/или её интерпретация может иметь множество недостатков: она может быть неоднозначной, неполной или вовсе неправильной.

Проблему установления корректности функционирования системы в какой-то мере решает сбор покрытия. Верификационное покрытие — это особая метрика, необходимая для оценки проверенного функционала относительно всего требуемого. Иначе говоря, покрытие оценивает прогресс и качество верификации. Следует отметить, что стопроцентное покрытие не гарантирует нам полной корректности работы проверяемого устройства. Например, в случае ошибки в спецификации или неполного верификационного плана все тесты могут проходить со стопроцентным покрытием, но устройство может содержать ошибки. Таким образом, информативным для нас является только неполное покрытие, так как оно указывает на части модуля, которые точно не проверялись.

3. Верификация должна выявлять ошибки как можно раньше.

Установлено, что с каждым последующим этапом разработки СнК стоимость исправления ошибки увеличивается. Основные финансовые потери возникают из-за задержки времени выпуска продукта относительно наиболее благоприятного окна возможностей на рынке (market window)[**applied_formal**]. Найти причину и устранить ошибку на ранних этапах разработки занимает значительно меньше времени, чем на этапах, более близких к отправке проекта на фабрику для печати.

Современные методы верификации можно поделить на динамические и статические методы; при этом из этих методов нельзя однозначно выделить наилучший. Их эффективность и даже применимость сильно зависят от характеристик проверяемого устройства, поэтому возникает потребность в изучении их применимости, качества и скорости по отношению к сложности и характерным особенностям проверяемой системы.

В связи с ростом популярности машинного обучения и активным развитием этой сферы сейчас с каждым годом растет потребность в графических картах общего назначения (GPGPU). Одним из ключевых компонентов графических карт является подсистема памяти, в частности, кэш второго уровня (L2 кэш). Основной сложностью верификации кэша является крайне большое количество состояний, которые может принять система.

1.1 Цель работы

Целью данной работы является изучение методов формальной верификации, трасс и методов, основанных на симуляции; исследование их применимости и эффективности в контексте кэша второго уровня GPGPU, а также сравнение этих методов.

2. Методы, основанные на симуляции

Одними из самых популярных методов функциональной верификации являются методы, основанные на симуляции (в дальнейшем будем называть такие методы просто симуляцией). Основная идея этих методов заключается в создании особого окружения (тестбенча[WTB]) вокруг проверяемого устройства (DUT) (см. рис. 1). Это окружение должно генерировать векторы входных данных, после чего подавать эти векторы на вход устройства. Одновременно с этим окружение производит независимые вычисления, преобразовывая входные векторы в соответствии со спецификацией. Далее окружение собирает выходной вектор с устройства и сравнивает его с предсказанным вектором.

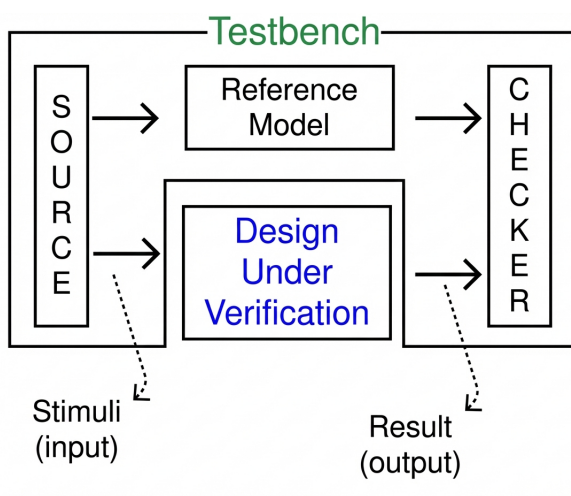


Рис. 1: Структура тестбенча

2.1 Направленное тестирование

Самым примитивным подходом динамической верификации является направленное тестирование. Используя его, инженер на основе спецификации составляет список тестов, каждый из которых фокусируется на одном отдельном свойстве устройства. На основе этого плана практически вручную прописываются входные векторы, которые тестируют заданные свойства. После успешного прохождения теста он вычеркивается из списка, и инженер переходит к следующему пункту.

Такая инкрементальность гарантирует почти мгновенные результаты. Тем не менее, этот подход является крайне немасштабируемым: количество тестов, которые необходимо разработать для верификации устройства, стремительно растет по мере его усложнения. Эту проблему можно смягчить, если использовать референсную модель для проверки устройства вместо написания индивидуальных тестбенчей для каждого теста.

Сейчас этот метод используется для проверки угловых случаев, которые трудно достичь при подаче случайных данных.

2.2 Ограниченное случайное тестирование

В настоящее время самый популярный подход — ограниченное случайное тестирование. При использовании данного подхода входные векторы не прописываются инженером самостоятельно, а генерируются случайно, хоть и в рамках заданных ограничений и весовых коэффициентов. Основной инженерной сложностью становится разработка автоматизированного способа предсказывать результат, который, как правило, представляет собой референсную модель устройства, написанную на более высоком уровне абстракции.

Изначальная задержка в получении результата ввиду построения референсной модели полностью компенсируется в дальнейшем: множество случайных входных векторов дают возможность находить ошибки гораздо быстрее, чем вручную прописанные вектора в направленном тестировании.

В связи с высокой масштабируемостью, удобством использования и быстротой получения результатов ограниченное случайное тестирование является доминирующим способом верификации уже многие годы. Ввиду этого было предложено и разработано множество библиотек для разработки верификационных окружений, например, OVM, VMM, UVM и др. Далее в работе при рассмотрении симуляции как метода верификации будем использовать UVM в виду её широкой распространённости.

2.2.1 UVM

UVM (Universal Verification Methodology)[UVM_IEEE][UVM_CB] — это стандартизированная методология построения тестовых окружений для функциональной верификации цифровых устройств.

Архитектура UVM(см. рис. 2) строится на компонентном подходе. Типичное тестовое окружение включает:

- Секвенсеры (sequencer) отвечают за генерацию входных векторов.
- Драйверы (driver) подают входное воздействие на DUT.
- Мониторы (monitor) снимают выходные векторы с DUT и передают их в scoreboard
- Агент (agent) объединяет в себе секвенсер, драйвер и монитор для взаимодействия с интерфейсами DUT. Как правило, число агентов равно количеству интерфейсов устройства.
- Система проверки результатов (scoreboard) генерирует ожидаемый ответ и сравнивает его с полученным от проверяемой системы ответом.
- Среда (environment) объединяет все вышеперечисленные компоненты в единый тестбенч.

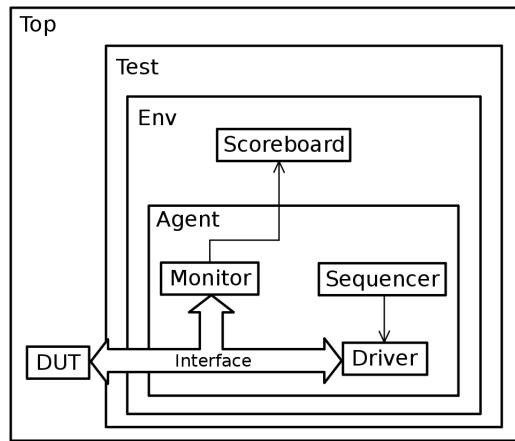


Рис. 2: Общая структура UVM окружения

UVM хорошо поддерживает ограниченное случайное тестирование и предоставляет инфраструктуру для построения сложных, повторно используемых и расширяемых систем функциональной верификации.

2.2.2 Функциональное и кодовое покрытие

При использовании методов, основанных на симуляции, принято выделять два основных вида покрытия[SVV][COV_CB]: кодовое (code coverage), показывающее, какие строки кода, ветви условий, выражения или состояния были выполнены в ходе симуляции, и функциональное (functional coverage), описывающее достижение заранее определённых сценариев и условий, соответствующих требованиям спецификации. Кодовое покрытие собирается автоматически инструментами верификации, а функциональное покрытие разрабатывается верификатором на основе верификационного плана. Инженер устанавливает диапазоны входных и выходных сигналов, а также их комбинации, которые должны быть проверены в тестах.

2.3 Трассы

Как правило, при разработке сложных микроархитектур вначале пишется не RTL-код, а потактовая модель разрабатываемого устройства на языке более высокого уровня(например, C++). С помощью такой модели удобно проводить анализ производительности разрабатываемого устройства на бенчмарках, так как симуляция их кода занимала бы нереалистичные промежутки времени на RTL симуляторе.

Вместе с тем, в процессе анализа формируются векторы входных и выходных данных, которые проходят через эту модель; их будем называть трассами (traces). Для проверки соответствия разработанного RTL-модуля мы можем использовать эти трассы как средство верификации: будем подавать участки записанных входных векторов на устройство и сравнивать выходные данные с референсными значениями. Практически таким же образом можем использовать чекпоинты (checkpoint) — снимки состояния

системы (например, памяти) в определённые моменты времени.

Этот метод позволяет быстро оценивать соответствие RTL-имплементации и референсной модели. Однако, поскольку потактовая модель разрабатывается на более высоком уровне, в ней теряются некоторые детали реализации, например, арбитрация выходных данных. Более того, такой метод верификации является нецелесообразным или вовсе неприменимым для блоков малых размеров.

2.4 Ключевые особенности

Симуляция имеет несколько ключевых особенностей. Во-первых, так как мы выборочно “обстреливаем” блок случайными данными, невозможно с уверенностью сказать, что блок работает корректно на основании одного успешно пройденного теста. Поэтому после разработки верификационного окружения и достижения требуемого уровня покрытия начинается этап регрессионного тестирования — регулярного автоматического запуска тестов. Регрессионное тестирование также позволяет автоматически отслеживать корректность поведения блока, если в него вводят изменения.

Во-вторых, наличие сложной комбинационной или последовательной логики, широких шин или массивов памяти не делает метод принципиально неприменимым. Это привносит важное качество универсальности: метод можно применять ко всем уровням абстракции, от мелких компонентов до полноценной СнК.

3. Формальная верификация

Формальная верификация относится к статическим методам верификации, то есть проверка корректности функционирования устройства производится без фактического запуска RTL-кода и моделирования его поведения во времени. Вместо этого к модели применяются формальные (математические) методы, с помощью которых доказываются, что устройство удовлетворяет (или не удовлетворяет) заданным в спецификации требованиям

3.1 Базовые принципы работы

Основой формальной верификации является алгоритм проверки модели (model checking algorithm)[**formal_roadmap**]. На вход этот алгоритм принимает проверяемую модель и формальную спецификацию, выраженную в виде темпоральных свойств. На выход алгоритм подаёт один из двух результатов: либо подтверждение того, что модель удовлетворяет спецификации, либо контрпример в случае нарушения свойства. Контрпример представляет собой временную диаграмму (вейвформу), демонстрирующую, каким образом происходит нарушение спецификации.

Кратко опишем алгоритм проверки модели при условии, что мы имеем формальное свойство ϕ и RTL-модуль M :

1. Создаем проверяющий автомат для $\neg\phi$.
2. Извлекаем из M конечный автомат J .
3. Считаем произведение J и ϕ .
4. Если произведение содержит какой-либо путь (то есть оно не пусто), то мы установили, что M содержит путь, который удовлетворяет $\neg\phi$, и, следовательно, опровергает ϕ . Таким образом, можно сообщить о наличии ошибки, а в качестве контрпримера привести найденный путь.
5. Если произведение пусто, то считаем, что M удовлетворяет ϕ .

Умножение вместе с проверкой на пустоту линейно относительно размеров автомата модели и проверяющего автомата. В связи с этим свойства, как правило, имеют небольшой размер, а более крупные свойства при возможности принято разбивать на несколько мелких во избежание роста сложности.

Таким образом, основную нагрузку представляет размер извлечённого из модели конечного автомата. Рассмотрим некоторые особенности, которые могут значительно влиять на сложность устройства:

- **Последовательная логика** дает сильное усложнение за счет резкого роста количества состояний. Например, для FIFO шириной W и глубиной D количество возможных состояний после сброса $N = 2^{W \cdot D}$. К тому же, усложняется запись временного свойства, что ведёт к более долгой разработке утверждений и усложнению проверяющего автомата.
- **Транспортировка данных** подразумевает наличие широких шин, очередей, памяти и др., что увеличивает сложность устройства.
- Для **трансформации данных** справедлив вывод из предыдущего пункта, но в то же время размер итогового конечного автомата увеличивается за счёт вычислений между битами данных.
- **Параметры** усложняют доказательство двумя путями. Во-первых, для каждого набора параметров нужен свой прогон утверждений, поэтому суммарное количество доказываемых свойств растёт (особенно если параметр — это не бинарная величина). Во-вторых, значение параметра может влиять на время, требуемое для доказательства, например, в случае FIFO с параметризованной глубиной.

При попытке применить формальную верификацию на практике можно очень быстро столкнуться с проблемой комбинаторного взрыва — стремительного роста времени, затрачиваемого алгоритмом, при увеличении сложности устройства.

3.2 Проблема комбинаторного взрыва

Полностью решить проблему комбинаторного взрыва невозможно: в худшем случае сложность извлечённого из модуля конечного автомата будет расти экспоненциально от сложности самого модуля. Тем не менее, существует ряд решений, которые существенно упрощают представление конечного автомата [state_exp_prob]. Рассмотрим наиболее популярные сейчас.

3.2.1 Бинарная диаграмма решений

BDD (Binary Decision Diagram) — это компактное каноническое представление булевых функций. Отметим, что, так как это представление каноническое (то есть такое, что для любых двух функционально эквивалентных функций оно будет одинаковым), его также используют и в проверке эквивалентности (Equivalence Checking).

Именно использование BDD вывело формальную верификацию в ряды практически применимых методов. Первые исследования показывали, что формальную верификацию можно применять для систем, содержащих вплоть до 10^{20} состояний [states_and_beyond], что на тот момент считалось недостижимым с использованием более классических методов.

Для BDD фундаментальной является теорема разложения Шэннона, которая гласит, что любую булеву функцию $f(x_1, x_2, \dots, x_k)$ от переменных $x_1, x_2, \dots, x_k$ мы можем представить в виде $f(x_1, x_2, \dots, x_k) = (x_1 \wedge f_{x_1=1}) \vee (\neg x_1 \wedge f_{x_1=0})$. Последовательно применяя эту теорему к логике, описывающей проверяемую модель, можем получить конечный автомат в виде дерева решений. Каждый узел этого дерева представляет булеву функцию поддеревья этого узла.

Последовательность, в которой мы выбираем переменные для разложения Шэннона, называется порядком переменных. При этом такое дерево решений называется упорядоченным, то есть различные переменные появляются в одном и том же порядке во всех путях от корневого узла графа до одной из терминальных вершин.

Некоторые узлы дерева решений могут выражать одну и ту же функцию. Пользуясь ациклическостью графа, можем использовать два правила для упрощения дерева:

1. Если два узла представляют одну и ту же функцию, то есть, они изоморфны, то произведем их слияние.
2. Если у узла два изоморфных потомка, то удаляем узел.

BDD можно строить и для последовательной логики. В таком случае, однако, вначале требуется преобразовать логику в булеву функцию путём введения дополнительных переменных для хранения текущего и следующего значений для каждого триггера в системе.

Таким образом, мы получаем сокращенное упорядоченное бинарную диаграмму решений, или же СУБДР (ROBDD). Чаще всего под термином BDD подразумевают именно эту форму.

Тем не менее, основным ограничением в использовании BDD все еще является размер. Хотя BDD и является более компактной формой записи булевых функций, она не устраняет проблему комбинаторного взрыва, а лишь смягчает её. К тому же, размер может сильно зависеть от порядка выбора переменных; в худшем случае — экспоненциально, а алгоритмы поиска наилучшего упорядочивания сами по себе могут быть сложными.

3.2.2 Задача выполнимости булевых формул

Как было показано ранее, доказательство на основе BDD может затрачивать огромное количество ресурсов. В то же время, для нахождения контрпримера к свойству нам необязательно выражать конечный автомат модели полностью. Обычно мы ожидаем, что свойство может быть нарушено в течение n первых тактов, поэтому нам достаточно просто развернуть конечный автомат до этого предела.

В таком случае проблему нахождения контрпримера можно свести к задаче выполнимости булевых формул (SAT-задаче) `[formal_roadmap][sat_formal]`. Действительно, пусть у нас есть первоначальное состояние I и свойство P . Введем свойство $\phi = \neg P$,

а также функцию смены состояний C . Наша задача — установить, можем ли мы найти след длины n такой, что удовлетворяется ϕ .

Для $n = 1$ проблема тривиальна: нам нужно установить, что изначальное состояние не противоречит спецификации, а это должно гарантироваться предположениями о системе.

Пусть $n = 2$. Создадим свойство-свидетеля $\phi \leftarrow Z$, то есть свойство, описывающее все возможные пути длины 2, при которых удовлетворяется ϕ . Тогда нам нужно решить SAT-проблему для следующей формулы:

$$I \wedge C^1 \wedge Z^1$$

Далее пусть $n = 3$. Снова "разворачиваем" конечный автомат, и получаем SAT-проблему для:

$$I \wedge C^1 \wedge C^2 \wedge Z^2$$

Таким образом, мы можем быстро найти контрпример. Следует также отметить, что SAT-решатели используются не только для нахождения контрпримеров, но и для доказательства корректности работы системы. Существует ряд случаев, когда методы, использующие BDD, принципиально неприменимы. В таком случае мы можем доказать свойство **ограниченно**, то есть только для первых нескольких тактов. Несмотря на то, что инструмент верификации не даст однозначного ответа, если свойство справедливо для числа тактов, в 2-3 раза превосходящих глубину системы, то можно считать свойство справедливым в целом [**bounded**].

Преимуществом данного подхода является существенное технологическое развитие в сфере SAT-решателей. Из-за распространённости проблемы она была глубоко изучена, и было выведено множество оптимизаций по её решению, в связи с чем современные SAT-решатели являются исключительно быстрыми.

3.2.3 Редукция конуса влияния

COI (Cone of Influence, конус влияния) [**formal_verif**] — это множество входных сигналов, комбинационной логики и регистров, которые *структурно* влияют на результат утверждения. Часто COI покрывает только часть всего модуля, то есть какие-то части блока никак не влияют на доказательство свойства. В таком случае мы можем извлечь из системы только релевантные компоненты и построить редуцированный конечный автомат. Очевидно, что если свойство справедливо для такого редуцированного автомата, то оно справедливо и для всей системы.

3.3 Формальное покрытие

Формальное покрытие[`formal_coverage`] принципиально отличается от покрытия в симуляции. При применении формальных методов устройство проверяется исчерпывающе, то есть проверяются все возможные комбинации входных данных одновременно, включая труднодостижимые в симуляции угловые случаи. Тем не менее, возникает другая проблема: если в симуляции мы могли недостаточно разнообразить данные, то в формальной верификации мы можем использовать лишние предположения, которые сделают все наши проверки бесполезными. Более того, требуется способ как-то оценить полноту набора проверяемых свойств, то есть какую часть устройства проверяют.

Для решения первой проблемы используется стимульное покрытие (`stimuli coverage`). При данном виде покрытия утверждения игнорируются, а значение имеют только предположения. При сборе этого покрытия проверяется, что любые легальные значения переменных могут быть достигнуты хоть при каком-то вводе, то есть блок не чрезмерно ограничен.

Для второй проблемы собирается покрытие проверок (`checker coverage`). Оно основывается на проверенных (как ограниченно, так и неограниченно) утверждениях и подразделяется на два типа: COI и Proof Core. Как уже было сказано ранее, COI — это структурно важная для утверждения логика. Proof Core (ядро доказательства) — это минимальное множество состояний, необходимое для доказательства утверждения. Покрытие проверок для обоих типов является объединением соответствующих метрик для всех проверяемых утверждений.

4. Используемые инструменты

4.1 JasperGold

JasperGold[**JASPER**] — это платформа для формальной верификации, представляющая собой набор приложений (Apps), каждое из которых соответствует определённой функции. Основным приложением JasperGold является FPV App (Formal Property Verification). Это приложение используется для доказательства свойств, нахождения контрпримеров, а также предоставляет интерфейс для отладки.

Работу FPV поддерживает ансамбль движков, основанных на вариациях BDD или SAT алгоритмов. Движки могут работать со всеми свойствами одновременно, выборочно с некоторыми или только с одним свойством. Чаще всего применяется соревновательный подход: над одним и тем же свойством одновременно работают несколько движков, а JasperGold в процессе доказательства определяет наиболее эффективный из них. Вследствие этого он может динамически перераспределять ресурсы, отключать или включать движки, а также регулировать их параметры.

Для измерения формального покрытия будет использоваться другое приложение — COV App (Coverage).

Также в дальнейшем нам понадобится оценивать сложность рассматриваемых устройств. Для этого будем использовать статистику, предоставляемую JasperGold в процессе анализа DUT, в частности, будем рассматривать количество используемых триггеров и логических вентилей.

4.2 SystemVerilog Assertions (SVA)

SystemVerilog Assertions — это языковые конструкции SystemVerilog, представляющие собой темпоральный язык для записи свойств.

Основой языка являются свойства. Свойство описывает поведение системы и может быть использовано как предположение, утверждение или спецификация покрытия в зависимости от используемой директивы: `assume`, `assert` и `cover` соответственно. Само по себе свойство никакого результата не производит.

Рассмотрим подробнее три директивы и их поведение в контексте формальной верификации:

- `assume()` — директива предположения, то есть свойства, которое будет ограничивать возможные состояния проверяемой системы. Инструмент формальной верификации считает предположение правдивым, как если бы оно было аксиомой, и в дальнейшем исследует только те пути, где все предположения справедливы. Чаще всего предположения — это свойства, обусловленные внешними факторами (протоколы, подсоединяемые блоки и др.).

- `assert()` — утверждение. Утверждение — это свойство, которое мы считаем всегда правдивым, и которое будет доказывать инструмент формальной верификации.
- `cover()` — это спецификация покрытия. Это свойство мы ожидаем быть справедливым хотя бы единожды. Оно играет одинаково важную роль с утверждением и является проверкой, что 1) блок способен достичь состояний, указанных в спецификации, и 2) своими предположениями мы не ограничили блок чрезмерно.

4.3 Характеристики машины

Приведем некоторые характеристики машины, на которой будут запускаться все тесты:

- Процессор: Intel(R) Xeon(R) Gold 5122 @ 3.60GHz
- Количество ядер: 16
- Оперативная память (RAM): 754 Гб

5. Практическое применение методов верификации к L2 кэшу GPGPU

L2 кэш в заданной имплементации представляет собой устройство, которое способно обслуживать 4 абонентов-TPC (Texture Processing Cluster) и содержит внутри себя 4 независимых партии. Приведем схему (см. рис. 3), демонстрирующую взаимодействие компонент партии между собой. DD-stage(Data Decode) принимает данные с L1 кэша и формирует пакеты, отправляемые в TLB (Translation Lookaside Buffer) и T-stage. T-stage(Tag) принимает запрос и определяет, отправлять ли его дальше. S-stage(Schedule) формирует "окончателное расписание", то есть действия, производимые над пакетом на последующих стадиях. Далее заявка отправляется в D-stage(Data); в этом блоке хранится массив данных кэша. На этой стадии данные могут записываться или читаться, также возможно проведение атомарных операций. После пакеты могут направляться на M-stage, Q-stage или C-stage (подробности распределения описаны в краткой спецификации D-stage). С C-stage формируется обратный ответ в L1 кэш, а с Q-stage отправляется заявка в Memory Crossbar, который после обращается непосредственно в память.

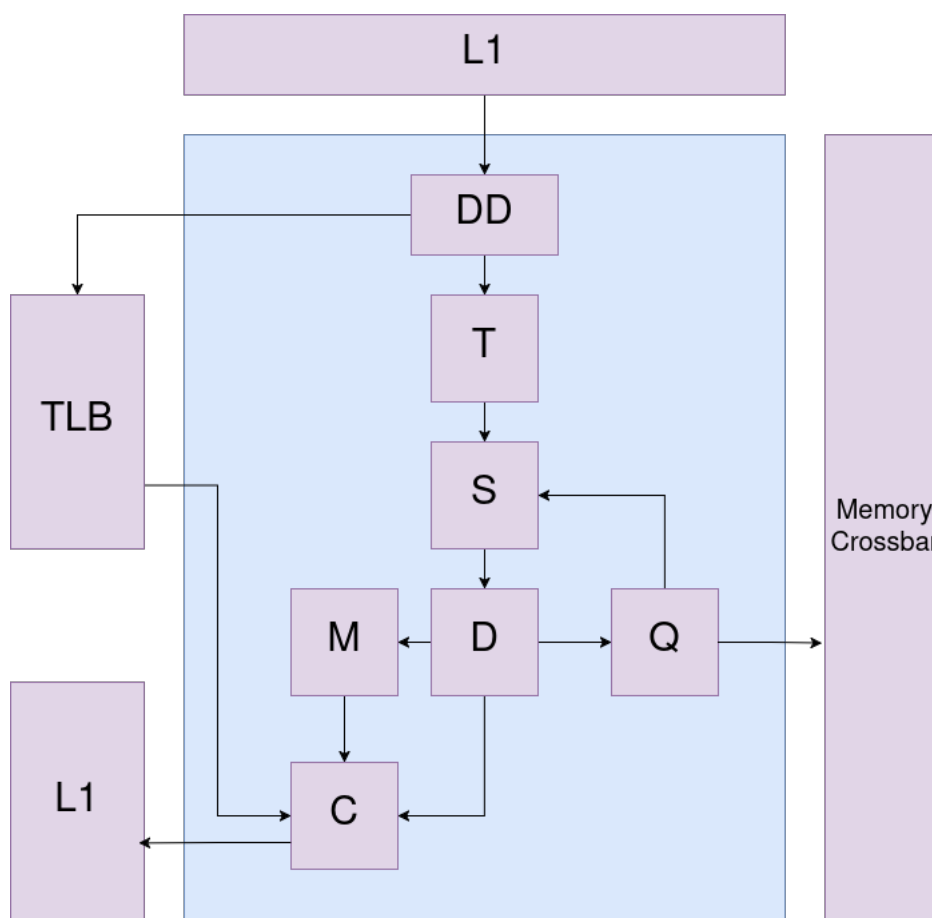


Рис. 3: Партия L2 кэша GPGPU

В контексте верификации нас интересуют следующие особенности кэша второго уровня:

- В кэше ведется активный обмен данными, из чего следует наличие широких шин внутри компонента.
- Наличие внутренних блоков памяти критически увеличивает количество состояний, которые может принять система.
- Практически отсутствует сложная вычислительная логика.

К кэшу целиком и отдельно партициям применим метод трасс. Блоки, составляющие партицию, верифицированы методом ограниченного случайного тестирования и формальными методами. Обратим внимание, что особенности кэша делают его особенно трудным для формальной верификации; в то же время методы, основанные на симуляции, не имеют в этом контексте отличительных особенностей. В связи с этим упор будет делаться именно на формальную верификацию, как на наиболее уязвимый метод.

Ниже рассмотрим отдельные блоки, входящие в состав партиции L2 кэша. Каждое из ниже рассмотренных устройств обладает своей отличительной особенностью, которая заметно усложняет формальную верификацию, поэтому будем уделять особое внимание различным техникам борьбы с ними. Будем указывать сложность систем, а также примерно время, затраченное на доказательство всех свойств, разработанных для блока. Также будем приводить анализ некоторых найденных ошибок.

5.1 D-stage

D-stage забирает инструкции с данными с S-stage, при необходимости записывает данные, читает их и/или выполняет атомарную инструкцию, а после отправляет выходной пакет на C-, M- или Q-stage. Атомарные инструкции могут выполняться над разными типами чисел, включая числа с плавающей запятой.

Некоторые пункты спецификации

- На всех интерфейсах valid-then-ready рукопожатие. Это подразумевает, что:
 - После того, как инициатор обращения поднимает сигнал валидности valid, он не опускает его до момента появления сигнала готовности ready со стороны устройства, к которому было совершено обращение
 - После поднятия сигнала valid передаваемые данные должны оставаться постоянными до прихода сигнала ready

- Операция MEMBAR проходит на интерфейс с C-stage без выполнения каких-либо действий.
- Операции STORE и UPDATE записывают данные в память, при этом STORE на выход ничего не подает.
- Операция LOAD читает данные из памяти и записывает их на интерфейс с C-stage.
- Операция ATOM читает данные из памяти, отправляет запрос в Atomic Unit, передаёт считанные данные на интерфейс с C-stage, после чего записывает в память результат выполненной операции

Верификация

Для примера приведём верификационное UVM окружение:

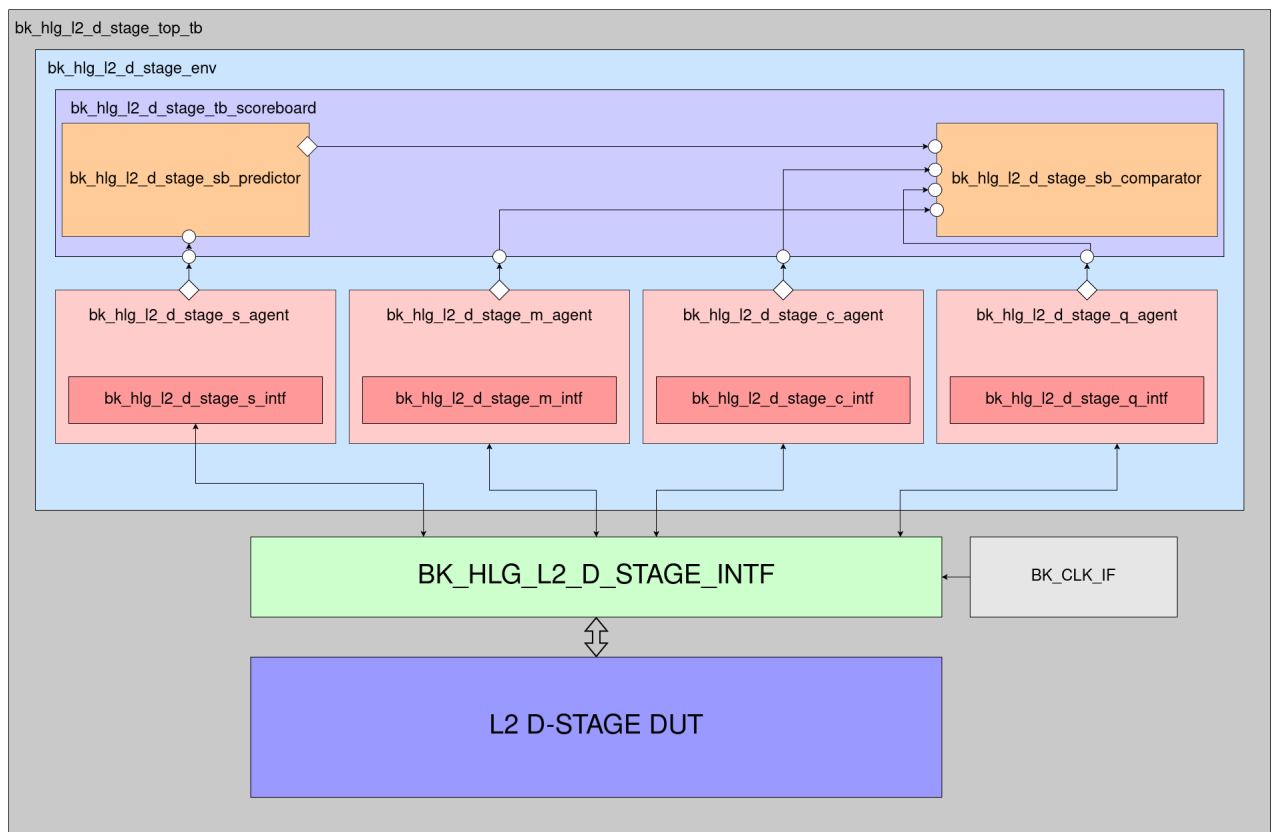


Рис. 4: Верификационное окружение D-stage

Также приведём график прогресса сбора кодового покрытия, используя ограниченное случайное тестирование, в зависимости от количества прошедших транзакций (см. рис.5). Обратим внимание на то, что процент кодового покрытия не достигает 100%: это вызвано наличием недостижимых состояний.

Основную сложность в верификации этого модуля представляет его объем и вычислительная нагрузка, связанная с атомарными операциями. Без каких-либо упрощений

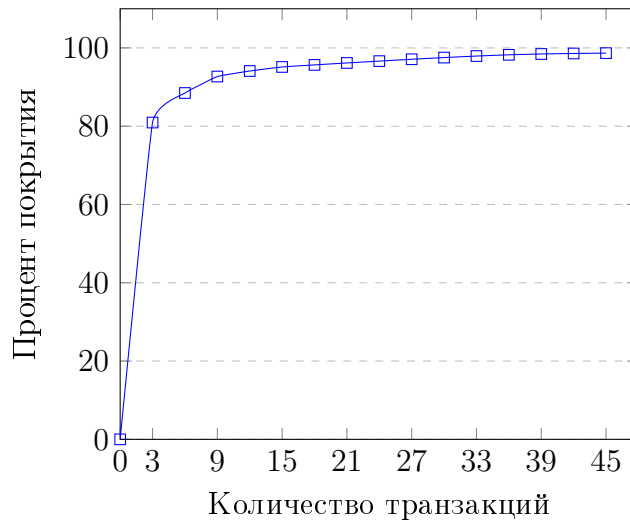


Рис. 5: Процент кодового покрытия в зависимости от количества транзакций для D-stage

инструмент способен доказать свойства только на 1 такт после сброса, поэтому для начала по очереди разберемся с этими проблемами.

Сначала попробуем решить проблему наличия памяти. Модуль содержит блок памяти 512x2048 бит, что находится за гранью любого современного инструмента формальной верификации. Можно было бы попробовать условно разделить модуль на две части: до и после памяти. В первой части мы будем проверять, что в память записываются правильные величины, подключаясь непосредственно к управляющим сигналам; во второй части будем проверять правильность выходных данных, подключаясь к сигналам, выходящим из памяти.

Второй подход заключается в замещении реальной памяти её моделью. JasperGold предоставляет для некоторых часто используемых, но сложных для формальной верификации устройств специальные блоки, являющиеся упрощёнными моделями этих устройств. Такие блоки будем называть акселераторами, согласно документации. Сам акселератор работает по следующему принципу: внутри себя он может хранить до 32 величин, соответствующих паре адрес-данные. Так как в процессе верификации блоков с памятью внутри одного свойства редко требуется больше одного-двух адресов, то такая упрощённая модель функционально ничем не отличается от реальной памяти, при этом сильно ускоряя процесс доказательства.

Самый главный минус акселератора — его крайняя степень негибкости. Сигналы, контролирующие запись и чтение, могут работать отлично от реализации в проверяемой модели, что требует дополнительных модификаций. В данном случае модификации оказались не слишком существенными; тем не менее, подключение акселератора к модели заняло значительную часть времени разработки проекта, что заметно отложило получение первых результатов.

Второй важной проблемой является наличие внутри логически простого блока устрой-

ства для проведения атомарных операций, включая операции над числами с плавающей запятой. Известно, что формальная верификация не подходит для верификации таких блоков, поэтому, ввиду отсутствия других решений, было принято решение подключаться только к входным и выходным сигналам модуля, не проверяя его внутреннюю логику. Таким образом, мы исключаем её из конуса влияния, чем ещё сильнее упрощаем верификацию.

Приведём примеры используемых свойств. Для сокращения размера листингов далее будем стараться приводить только содержание всех свойств, без директив и объявлений. Также будем по необходимости оставлять поясняющие комментарии.

Для valid-then-ready рукопожатия напомним свойства следующего вида:

```

1      // операция не перестает быть валидной, пока её не примут
2      vld && !rdy | => vld;
3
4      // данные не изменяются, пока операцию не примут
5      vld && !rdy | => $stable(data);
6
7      //если транзакция пришла на интерфейс, то когда-нибудь она будет
      обработана
8      vld && !rdy | => s_eventually (vld && rdy);

```

Операция MEMBAR не затрагивает памяти, поэтому для неё можем записать тривиальные свойства наподобие следующего:

```

1 property p_membar_opcode;
2     logic [OPCODE_W-1:0] v_opcode;
3     (s_valid_membar_instr, v_opcode = i_opcode) | => (o_c_opcode ==
        v_opcode);
4 endproperty

```

Здесь вместо использования локальной переменной `v_opcode` мы могли бы имплементировать буфер для временного хранения переменной. Однако такие буферы существенно усложняют доказательство за счёт наличия дополнительной логики и являются менее нативными для инструментов формальной верификации.

Для передаваемых в Atomic Unit данных свойства имеют аналогичную форму. Для сигнала валидности запишем следующее свойство:

```

1 $past(s_valid_atomic_instr.triggered) == i_atom_vld;

```

Здесь существенно использование оператора `(==)` равенства вместо импликации `(|>)`; это станет важным при рассмотрении найденных ошибок. Пока что отметим, что можно разделить это свойство на два более простых для дальнейшей оптимизации, так как функция `$past()` немного усложняет доказательство.

Гораздо сложнее записываются свойства, затрагивающие память. Выпишем свойство, выражающее read-after-write согласованность:

```
1 property p_load_after_store;
2     logic [DATA_W-1:0] v_data;
3     (s_valid_store_instr, v_data = i_data)
4     ##1 s_no_valid_write_instr throughout (s_valid_write_load)[->1]
5     |=> o_c_data == v_data;
6 endproperty
```

Для чтения после атомарной операции свойство становится ещё сложнее, хотя и имеет схожий вид:

```
1 property p_atom_after_store;
2     logic [DATA_W-1:0] v_data;
3     s_valid_store_instr
4     ##1 s_no_valid_write_instr throughout (atom_res_vld[->1], v_data
5     = atom_res_mem)
6     ##1 s_no_valid_write_instr throughout (s_valid_write_atom[->1])
7     |=> o_c_data == v_data;
8 endproperty
```

Анализ JasperGold показал, что блок содержит 374086 триггеров и 1074060 логических вентилях; при этом время доказательства для свойств составило 4694 минут (около 3 суток). Отметим, что свойства, затрагивающие память, удалось доказать только для первых 20-21 циклов (то есть ограниченно).

Найденные ошибки и их анализ

1. Неправильная передача данных в Atomic Unit: операция АТОМ может писать свой результат в адрес следующей инструкции, если она пришла раньше, чем закончила выполняться АТОМ. Эту ошибку обнаружила и симуляция, и формальная верификация.
2. Если пакет, обращающийся к Atomic Unit, несколько тактов не сменяется другим пакетом, то D-stage непрерывно обращается к Atomic Unit. Эта ошибка была выявлена только формальной верификацией. Это вызвано тем, что лишние обращения никак не изменяли результирующее значение в памяти, а следовательно, при попытке чтения таких данных никаких ошибок не возникало.
3. Самой серьёзной ошибкой оказалось отсутствие проверки на валидность инструкции при поступлении STORE/UPDATE, в следствие чего могла произойти запись данных из невалидного пакета. Эта ошибка была не сразу обнаружена симуляцией, так как в окружении присутствовал дефект: после инвалидации пакет не

изменялся вплоть до поступления следующего валидного пакета. Формальная верификация, тем временем, сразу обнаружила ошибку.

5.2 C-stage

C-stage забирает данные с D-stage, M-stage и TLB, и передает их в один из четырех выходных каналов.

Спецификация

- Входы организованы по valid-then-ready рукопожатию.
- Недопустим одновременный приём с более чем одного входа
- Выбор данных между блоками D-stage и M-stage происходит по Round Robin принципу
- Выбор данных между блоками TLB и D-stage или M-stage происходит по приоритетному принципу (TLB более приоритетный).
- Входные пакеты распределяются по одному из четырёх выходных интерфейсов в соответствии с номером, содержащимся в пакете
- Сигнал `o_pkt_cnt` отвечает за число пакетов, содержащихся в очереди соответствующего выходного интерфейса

Верификация

Прогресс сбора покрытия представлен на рис. 6.

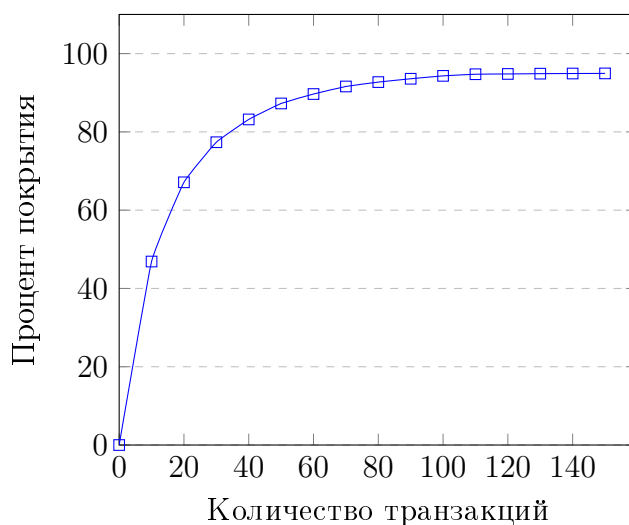


Рис. 6: Процент кодового покрытия в зависимости от количества транзакций для C-stage

Передачу данных верифицируем по аналогии с D-stage, то есть просто переиспользуем уже разработанные свойства. Отметим, что такое переиспользование — это очень эффективная и полезная практика, так как она позволяет существенно сократить время на полную верификацию проекта.

Приведем некоторые свойства для проверки арбитража.

```

1 // Принимаем данные только с одного входа
2 $onehot({s_d_tr.triggered, s_m_tr.triggered, s_tlb_tr.triggered});
3
4 // Приоритет TLB
5 i_tlb_vld |-> (!o_m_rdy && !o_d_rdy) until_with o_tlb_rdy;
6
7 // Приоритет M после D-транзакции
8 s_d_tr ##1 i_m_vld[->1] |-> !o_d_rdy until_with o_m_rdy;
9
10 // В какой-то момент произойдет обработка TLB запроса
11 i_tlb_vld && !o_tlb_rdy |=> o_tlb_rdy;
```

Особую сложность для формальной верификации представляют счетчики. В нашем случае он небольшой, и можно было бы его верифицировать с помощью другого, референсного счетчика, но гораздо экономнее будет проверить условия инкремента, декремента и отсутствия изменений. Преимущества такого подхода проявляются сильнее на крупных счётчиках. Пример свойства для инкремента:

```

1 logic [CNT_W-1:0] v_cnt;
2 (s_data2tpc[i] && !(o_vld[i] && i_rdy[i]), v_cnt = cnt[i]) |>
  cnt[i] == v_cnt + 1;
```

Для проверки корректной передачи данных напомним свойство сбалансированности:

```

1 generate
2   for (genvar i = 0; i < NUM_TPC; i++) begin
3     for (genvar j = 0; j <= FIFO_DEPTH; j++) begin
4       property p_d_data;
5         logic [DATA_W-1:0] v_data;
6         (s_d_tr and (i_d_route_info == i) and (cnt[i] == j),
          v_data = i_d_data) ##0 (o_vld[i] && i_rdy[i])[->j+1] |-> o_data ==
          v_data;
7       endproperty
8     end
9   end
10 endgenerate
```

Сложность C-stage составляет 22457 триггеров и 142640 вентилей; на полное доказательство ушло 1031 минута (около 17 часов).

Найденные ошибки и их анализ

1. Неправильная арбитрация: при определенных условиях мог быть выбран клиент с более низкой приоритетностью
2. Также найдена ошибка в bypass буфере, содержащемся в устройстве. Причиной ошибки стало то, что сигнал о том, что буфер полон, не сохранялся, когда данные не отправлялись в или из буфера

5.3 DD-stage

Спецификация

- Направляет операцию TLBREQ в TLB
- Направляет операции STORE/ATOM/RED/MEMBAR/FLUSH/INVAL в T-stage
- Делит LOAD на 1-4 LOADa и направляет в T-stage
- Команды FLUSH/INVAL приводят к отправке 2048 пакетов FLUSH/EVICT в T-stage. Новые пакеты не принимаются, пока не пришло 2048 (строго) ответов от T-/S-/Q-stage.

Верификация

Прогресс сбора покрытия представлен на рис. 7.

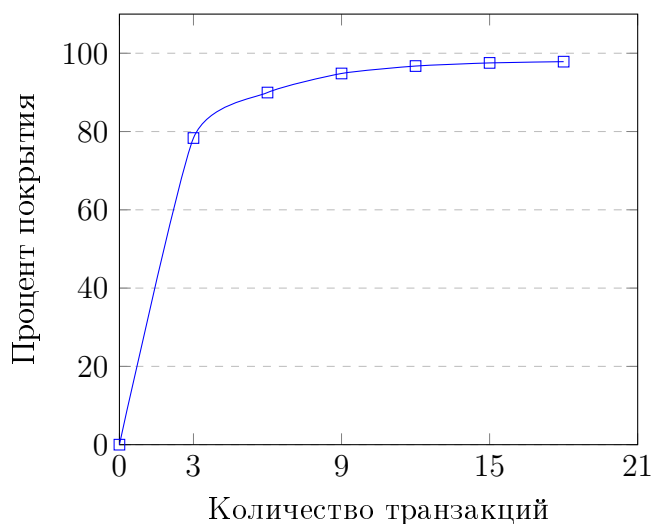


Рис. 7: Процент кодового покрытия в зависимости от количества транзакций для DD-stage

Верификация счетчика выполняется аналогично предыдущему блоку. Единственное отличие — теперь счетчик может инкрементироваться на 1, 2 или 3. Для упрощения

разделим на 3 свойства, каждое из которых будет проверять инкремент на соответствующий момент.

```

1 sequence s_num_ack(num);
2     $countones({t_ack, s_ack, q_ack}) == num_of_acks;
3 endsequence
4
5 // s_valid_ack_cnt -- секвенция валидности счетчика
6 property p_load_after_store(logic [ACK_CNT_W-1:0] num;
7     logic [ACK_CNT_W-1:0] v_ack_cnt;
8     (s_valid_ack_cnt(num) and s_num_ack(num), v_ack_cnt = ack_cnt)
9     | => (ack_cnt == v_ack_cnt + num);
10 endproperty

```

Особую сложность представляет верификация деления операции LOAD, так как количество операций зависит от флага внутри пришедшего LOAD, то есть является динамическим. В таком случае применяется подход генерирования утверждений, внутри которых в том или ином виде содержится сравнение с переменной итерации. Для поднятия сигнала o_rdy после отправки всех LOAD напомним следующее свойство:

```

1 generate
2     for (genvar i = 1; i < LOAD_MASK_W; i++) begin
3         property p_load_o_rdy_after_number_of_t_transactions;
4             (s_valid_load and ($countones(i_load_mask) == i) |-> (
5                 o_t_vld && o_t_rdy && (o_t_opcode == LOAD)[->i]) |-> o_rdy;
6             endproperty
7         end
8     endgenerate

```

Проверка сериализации данных выполняется несколько сложнее. В исходном пакете присутствует 4 поля для максимально возможного количества выходных пакетов, и в каждом из полей записаны данные, соответствующие конкретному выходному пакету. Необходимо проверить запись данных, учитывая то, что пакетов до рассматриваемого может быть любое количество. Для этого запишем следующее свойство:

```

1 generate
2     for (genvar i = 0; i < LOAD_MASK_W; i++) begin
3         for (genvar j = 1; j <= i+1; j++) begin
4             property p_load_order;
5                 (s_valid_load and i_load_mask[i] and ($countones(
6                     i_load_mask) == j) |-> (o_t_vld && o_t_rdy && (o_t_opcode == LOAD)
7                     [->j) |-> (o_t_vld && o_t_data == i_load_data[i]));
8             endproperty
9         end
10    end

```

DD-stage является наиболее лёгким для формальной верификацией блоком из всех рассмотренных в этой работе: он содержит 1788 триггеров и 34599 вентилей, а на полной доказательство ушло 46 минут.

5.4 T-stage

T-stage управляет наличием кэш-линии в кэше и её запираением для атомарных операций. При этом выбор way для соответствующего set осуществляется согласно алгоритму PLRU (Pseudo Last Recently Used).

Детали имплементации и верификации

Прогресс сбора покрытия представлен на рис. 8.

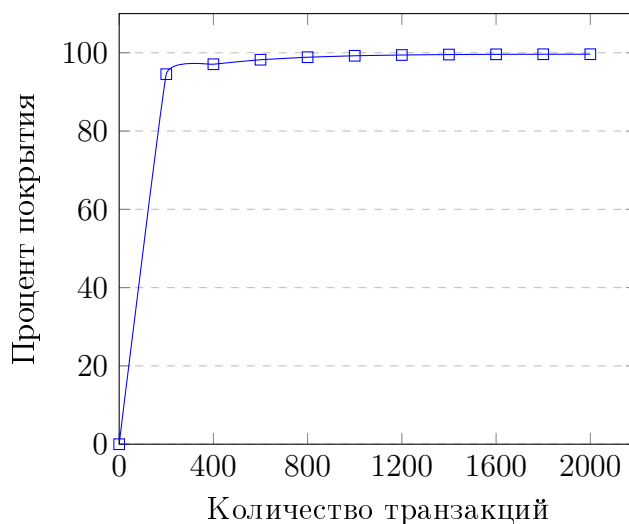


Рис. 8: Процент кодового покрытия в зависимости от количества транзакций для T-stage

T-stage можно условно разделить на блоки PLRU, T0, T1 и память. Все эти блоки по отдельности взаимодействуют со сложными последовательными зависимостями, поэтому писать свойства для T-stage целиком будет нецелесообразно по нескольким причинам. Во-первых, на практическом опыте можно установить, что чем больше свойство, тем труднее его правильно реализовать (написать). Во-вторых, даже если верификатор располагает временным ресурсом для написания такого сложного свойства, для инструмента формальной верификации оно может оказаться слишком сложным, и оно может вообще не скомпилироваться. Даже для компиляции гораздо более простых временных свойств DD-stage автору было необходимо повысить автоматически установленный таймаут компиляции.

Поэтому можем сделать вывод, что верифицировать блок целиком нецелесообразно, так что следует применить технику разделения блока на более мелкие части.

Такое разделение следует производить крайне осторожно, так как мы подключаемся к внутренним сигналам, и существует опасность пропустить какой-либо из них. Поэтому при разделении блока для каждой части следует каждую из них рассматривать как цельный блок. Свойства должны быть исчерпывающими для каждого выходного сигнала каждой из частей; в этом помогает наличие в спецификации схемы устройства. Однако отметим, что предположения, если таковые используются, должны применяться только к входным сигналам целого блока, ни в коем случае не к его внутренним сигналам.

В целом, T-stage можно признать наиболее непригодным для формальной верификации из всех рассмотренных в работе блоков. Сложная последовательная логика и вычисления PLRU сильно усложняют задачу верификации, несмотря на то, что общее число состояний системы сравнительно мало, и после разбиения все разработанные свойства гораздо проще рассмотренных ранее. Это приводит нас к выводу, что применимость формальной верификации является куда более комплексным вопросом, чем простой подсчёт сложности модели.

При сложности в 79936 триггеров и 281233 логических вентилей, на полное доказательство ушло 3752 минут (около 2,5 суток).

5.5 Q-stage

Детали имплементации и верификации

Прогресс сбора покрытия представлен на рис. 9.

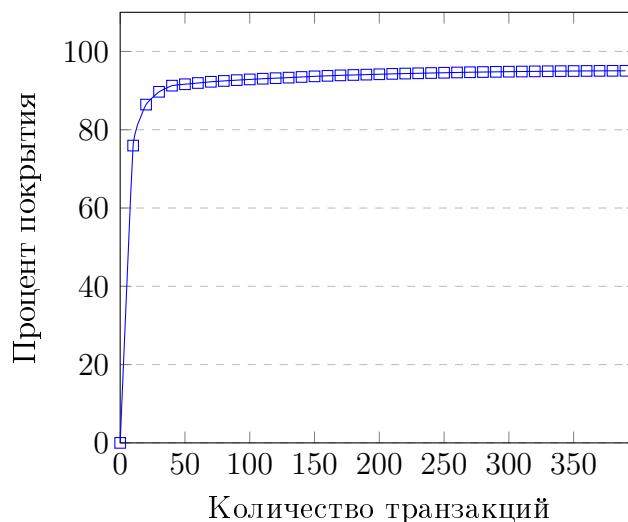


Рис. 9: Процент кодового покрытия в зависимости от количества транзакций для Q-stage.

Q-stage буферизирует запросы в очередь `req_q`, после чего кладёт их в очередь ожидания ответа: в одну из 4 `miss_lookup_q` и/или `flush_lookup_q`. Также буферизирует прочитанные данные в `update_q` очереди.

Очевидно наличие большого количества очередей, что сильно увеличивает последовательную глубину устройства. Поэтому ко всему блоку применим метод разделения блока на более простые подмодули, изолируя сложные для верификации очереди, как уже было рассмотрено в примере T-stage. Тем не менее, для полной верификации устройства нам все еще необходимо написать отдельную группу свойств для используемых FIFO.

Самым примитивным подходом будет построить свою референсную модель FIFO и использовать её для сравнения с имплементацией. Так как размеры FIFO в RTL, как правило, небольшие (в случае данного блока глубина всех FIFO равна 8), то такой подход достаточно распространён и считается классическим[[sva_handbook](#)]. Однако мы можем немного оптимизировать задачу, вместо FIFO используя счётчик хранимых в нём данных:

```

1 always_ff @(posedge clk) begin
2     if (!rst_n) begin
3         v_diff <= 0
4     end
5     else begin
6         case ({wr_trans, tr_trans})
7             2'b10: v_diff <= v_diff + 1;
8             2'b01: v_diff <= v_diff - 1;
9         endcase
10    end
11 end

```

Тогда, например, свойство для корректной передачи данных имеет вид:

```

1 generate
2     for (genvar i = 1; i < 2**ADDR_W; i++) begin
3         property p_data;
4             (wr_tr and (v_diff == i), v_data = i_wr_data) ##0 rd_tr
5             [->(i+1)] |-> o_rd_data == v_data;
6         endproperty
7     end
8 endgenerate

```

А для исходящих сигналов vld и rdy:

```

1 o_rd_vld == (v_diff != 0);
2
3 o_wr_rdy == (v_diff != 2**ADDR_W);

```

В целом, использование дополнительной логики для такой задачи неизбежно из-за невозможности адекватно выразить спецификацию исключительно в свойствах. Мо-

жет показаться, что дополнительная логика увеличивает сложность доказательства свойств. Отчасти это утверждение справедливо: действительно, реализовывать полную референсную модель устройства крайне неразумно: так количество состояний всей системы увеличивается вдвое. Тем не менее, существует ряд случаев, где дополнительная логика существенно увеличивает производительность за счёт уменьшения размера итогового проверяющего автомата.

Сложность Q-stage: 35906 триггеров и 195514 вентиляей; на полное доказательство ушло 720 минут (12 часов).

6. Сравнение примененных методов верификации

На основе полученных данных проведем комплексное сравнение рассмотренных методов верификации, особое внимание уделяя различиям между симуляцией и формальной верификацией. По умолчанию выводы делаются для устройств, для которых применяется формальная верификация, если не оговорено иное.

6.1 Прогресс верификации

Для оценки прогресса верификации выберем два основных параметра:

1. Время, затрачиваемое от начала разработки устройства до обнаружения ошибки.
2. Время, затрачиваемое на полную верификацию устройства.

Разумеется, точные цифры зависят от многих факторов, таких как опыт верификатора, сложность блока, качество спецификации, вычислительные ресурсы и др. Однако на основе практического применения и общих соображений мы можем качественно сравнить два метода.

Если целью является как можно быстрее обнаружить ошибку, то, как правило, лучше использовать формальную верификацию. Приведем несколько аргументов:

- В отличие от симуляции, разработку формального проекта можно начинать даже до завершения модуля RTL-инженером, ведь набор формальных свойств — это и есть спецификация, просто записанная на формальном языке. При этом утверждения легко можно включать или отключать, чтобы сосредоточиться на одной части дизайна.
- Судить о корректности работы блока при использовании симуляции можно лишь при почти завершившейся разработке тестбенча и последовательностей. Формальная верификация обладает инкрементальностью, поэтому результаты можно увидеть почти мгновенно. Этому также способствует возможность переиспользовать свойства для разных блоков (например, для `valid-then-ready handshake`).

Для полной верификации устройства ответ разнится в зависимости от того, где мы устанавливаем границу уверенности в том, что устройство работает корректно. Основываясь на практическом опыте, можно сказать, что разработка формальных свойств занимает время, сопоставимое с разработкой верификационного окружения. Тем не менее, симуляция занимает гораздо меньше времени на проведение всех тестов и сбор покрытия (минуты по сравнению с часами или днями). В связи с этим первые предположения о корректном поведении системы у динамических методов появляются заметно раньше.

В качестве подтверждения сведём в таблицу 1 данные, полученные в ходе применения методов формальной верификации и симуляции:

Название блока	D-stage	C-stage	DD-stage	T-stage	Q-stage
Количество триггеров	374086	22457	1788	79936	35906
Количество логических вентилей	1074060	142640	34599	271233	195514
Время, затраченное на формальную верификацию, минут	4694 (ограниченно)	1031	46	3752	720
Количество транзакций, необходимое для сбора 100% покрытия	45	150	18	2000	390
Время, затрачиваемое симуляцией для сбора полного кодового покрытия, секунд	10	12	10	201	19

Таблица 1: Время, затраченное на верификацию блоков используя методы формальной верификации и UVM-тестирования.

Тем не менее, симуляция не способна предоставить исчерпывающий ответ в какие-либо разумные временные рамки. Пусть мы имеем блок сложностью примерно 10^{20} (около 2^{66} - 2^{67}) состояний и симулятор, который способен симулировать 1 миллиард тактов в секунду, что сильно превышает возможности современных симуляторов. В таком случае на проверку всех возможных состояний нам потребуется около 3 тысяч лет. В то же время формальная верификация предоставила ответ для D-stage, верхнюю границу сложности которого можно оценить в 2^{374086} состояний, в течение 3 суток.

6.2 Качество верификации

В целом, для систем среднего размера можно сказать, что ни один из методов не может обеспечить полную верификацию устройства, так как все они упираются в вычислительные ресурсы: проверка всех возможных состояний симуляцией занимает, как показано ранее, миллионы лет, а формальные методы будут требовать слишком много времени и памяти, в связи с чем придется так или иначе упрощать устройство.

Тем не менее, здесь формальная верификация обладает существенным преимуществом. Под заданными предположениями инструмент проверяет всё пространство возможных состояний системы, включая сложные для обнаружения симуляцией угловые случаи. Иначе говоря, устройство проверяется *исчерпывающе*.

6.3 Исправление ошибок

Часто обнаружить ошибку бывает гораздо легче, чем найти её причину, поэтому стоит обратить внимание на то, в какой форме метод сообщает об ошибке и какие возможно-

сти для её исправления ("дебага") он даёт.

Симуляция и в случае успеха, и в случае провала предоставляет фиксированную, очень длинную (тысячи или сотни тысяч тактов) вейвформу с большим числом нерелевантной информации. Это может быть полезно для визуализации поведения блока, однако для обнаружения причины ошибки крайне неудобно. Симуляция сообщает, что ожидаемые и реальные выходные данные не совпадают, но не сообщает, почему. Более того, ошибка может возникать настолько редко, что возможность воспроизвести её при каких-либо других входных данных становится маловероятной.

С другой стороны, формальная верификация почти всегда предоставляет кратчайший путь к достижению ошибки. Современные инструменты формальной верификации указывают на логику, влияющую на результат (COI, Proof Core), а также позволяют изменить вейвформу для изучения поведения блока. Таким образом, улучшается локальность обнаружения ошибки.

6.4 Порог входа

Порог входа в симуляцию может быть разным в зависимости от подхода: так, например, использовать направленное тестирование гораздо легче, чем ограниченное случайное. Тем не менее, задачу сильно облегчает наличие широко используемой библиотеки UVM, и интуитивность подхода.

У формальной верификации порог входа на порядок выше. В первую очередь это вызвано строгими требованиями к записи спецификации вместе с необходимостью глубокого изучения SVA. От верификатора требуется чёткое понимание того, как именно преобразовать спецификацию в набор свойств, чтобы он был достаточным для проверки модели. При упрощении устройства требуется особая осторожность, чтобы не ограничить пространство состояний слишком сильно и не пропустить ошибку. Также зачастую требуется опыт в оптимизации свойств, которая сама по себе представляет отдельную подзадачу.

6.5 Применение

На основании теоретического исследования и практических результатов можем составить некоторый список практических рекомендаций по применению различных методов верификации. В целом, их можно свести к грамотному распределению ресурсов: формально верифицировать можно и сложные блоки, но на разработку свойств и их доказательство уйдёт очень много времени и вычислительных мощностей, не говоря о том, что свойства будут доказаны лишь для первых нескольких тактов. В то же время, где это возможно, хотелось бы иметь полную уверенность в корректности поведения устройства.

Поэтому в первую очередь следует проверить сложность модели. Если блок большой (сложность выходит за пределы сотен тысяч триггеров), то нет смысла рассматривать применение формальной верификации и стоит ограничиться симуляцией. Более того, если блок обладает достаточно высоким уровнем абстракции (в контексте данной работы это может быть кэш целиком или одна из его партиций), то следует рассмотреть метод трасс в дополнение к ограниченному случайному тестированию.

Если блок имеет средний размер (условно до 100000 триггеров) и небольшую глубину, то имеет смысл рассмотреть комбинацию формальной верификации и симуляции. Формальная верификация позволит удостовериться в корректности работы если не всего блока целом, то хотя бы его критически важных частей, где нам важно удостовериться в отсутствии ошибок. Симуляция же является более традиционным методом для таких устройств, и позволяет закрепить результат.

Для малых блоков (до 10000 триггеров) целесообразно использовать только формальную верификацию. Написание свойств для таких блоков навряд ли будет занимать более нескольких дней, как и их проверка. В то же время симуляция на небольших системах может показывать неудовлетворительные результаты: разработка окружения и сбор полного покрытия будут занимать непропорционально много ресурсов.

В общем случае, во время разработки модуля, когда верификация выполняется RTL-разработчиком, лучше использовать направленное тестирование и/или разработать тривиальные формальные свойства. Оба этих метода обеспечивают быстрые результаты и позволяют провести поверхностное тестирование.

Между подтверждением первоначальной функциональности модели и окончательной "заморозкой" (завершением разработки) RTL может пройти очень много времени. В этот период велика вероятность изменений в блоке, его полного переписывания или удаления. Полную формальную верификацию проводить можно, но проверка всех свойств может занимать до одной недели. Симуляция, с другой стороны, быстрее выдаст первые результаты: уже в первый день после внесения изменений можно приблизительно судить о наличии ошибок в блоке.

В конце цикла верификации, по возможности, лучше разработать хотя бы несколько наиболее важных свойств блока, особенно для критических компонентов. Это даст высокую степень уверенности в корректности функционирования модели.

Для часто используемых блоков и/или блоков небольшого размера (FIFO, буферы, арбитры, счётчики и др.) крайне рекомендуется использовать формальную верификацию как основной метод. Обычно такие блоки плохо покрываются симуляцией, когда они находятся в составе других модулей, и ошибки в них могут очень долго не проявляться в регрессии. Так, например, на ранних стадиях изучения формальной верификации автором была найдена ошибка в буфере, при том что регрессионное тестирование блока, содержащего этот буфер, до этого проходило успешно.

7. Заключение

В работе были изучены актуальные методы верификации, исследованы их преимущества и недостатки. На примере кэша второго уровня графического процессора общего назначения были исследованы такие характеристики, как качество верификации, скорость нахождения ошибок и полнота. На основе полученных результатов были сформулированы практические рекомендации по применению соответствующих методов.

Особое внимание было уделено формальной верификации как относительно новому и слабо освещённому методу. Было доказано, что этот метод можно эффективно применять к блокам среднего размера, в том числе содержащим сложные для формальных методов элементы, такие как память, счётчики, FIFO и др. Были показаны техники и свойства, с помощью которых был достигнут результат.

В будущем развитие данной работы может быть направлено на более углубленное изучение формальной верификации и пределов её применимости. Требуется дополнительное изучение возможных оптимизаций свойств, методов разбиения сложных блоков и подходов к сокращению пространства состояний. Кроме того, представляет интерес расширение исследования на другие компоненты кроме L2 кэша, что позволит уточнить практические границы эффективности рассмотренных методов и выработать более точные рекомендации по их применению в проектах различной сложности.